

《操作系统课程设计》A 题结题报告

项目题目：OS 内核实现

基础平台：xv6-riscv

课程名称	操作系统课程设计
报告性质	结题报告（方案 A）
小组成员	侯奕明、涂凡琛、韩硕
当前选题	A 题：OS 内核实现
提交时间	2026 年 6 月

2026 年 6 月 20 日

目录

一、项目概述	4
1.1 项目背景	4
1.2 项目目标	4
1.3 团队信息与分工方案	4
1.4 项目完成情况概览	5
二、技术方案设计	7
2.1 整体架构	7
2.2 模块划分	7
2.3 技术选型	8
三、模块详细实现	9
3.1 中断与异常处理（侯奕明）	9
3.1.1 设计思路	9
3.1.2 系统调用全链路	9
3.1.3 中断处理的两条路径	10
3.1.4 时钟中断机制	11
3.2 系统调用接口实现（侯奕明）	12
3.2.1 ps 系统调用	12
3.2.2 halt 关机系统调用	13
3.2.3 lseek 文件定位系统调用	14
3.2.4 Socket 网络系统调用	15
3.3 设备驱动与 I/O（侯奕明）	16
3.3.1 控制台输出原子化	16
3.3.2 virtio-net MMIO 网卡驱动	18
3.4 Shell 功能增强（侯奕明）	19
3.4.1 Shell PATH Fallback	19
3.4.2 halt 内建命令	20
3.5 网络协议栈实现（侯奕明）	21
3.5.1 总体架构	21
3.5.2 Ethernet / ARP / IP / UDP 协议实现	21
3.5.3 Socket 层设计	22
3.5.4 QEMU 网络配置	23
3.5.5 调试历程总结	23
3.6 进程管理与调度（涂凡琛）	23
3.6.1 FCFS 与 MLFQ 调度算法实现	23

3.6.2	信号量机制	24
3.6.3	wakeup 哈希队列优化	24
3.7	系统启动 (韩硕)	25
3.7.1	启动链路	25
3.7.2	关键初始化步骤	25
3.7.3	课程要求对应与实现结果	25
3.8	内存管理 (韩硕)	26
3.8.1	物理页框分配与启动时内存探测	26
3.8.2	虚拟内存与按需分页	27
3.8.3	内核堆分配器与课程指标	27
3.9	文件管理 (韩硕)	28
3.9.1	文件描述符表管理	28
3.9.2	文件与目录基本操作	29
3.9.3	路径解析与 seek 支持	29
四	系统运行与功能测试	30
4.1	基础系统运行验证	30
4.1.1	系统启动	30
4.1.2	ps 系统调用测试	30
4.1.3	halt 关机测试	30
4.1.4	lseek 测试	30
4.2	控制台输出原子化测试	31
4.2.1	修复前 vs 修复后	31
4.3	管道通信测试	31
4.4	Shell PATH Fallback 测试	32
4.5	UDP 网络通信测试	33
4.5.1	测试环境	33
4.5.2	Test 1: xv6 发送 → 宿主机接收	33
4.5.3	Test 2: 宿主机回复 → xv6 接收	33
4.6	调度算法测试 (涂凡琛)	33
4.6.1	FCFS 测试	33
4.6.2	MLFQ 测试	33
4.6.3	信号量测试	34
4.7	内存分配器测试 (韩硕)	34
4.8	文件管理测试 (韩硕)	34
五	创新点	36
5.1	UDP 网络协议栈 (侯奕明)	36

目录	3
5.2 控制台输出原子化的两层协同方案（侯奕明）	36
5.3 QEMU Test Device 关机方案（侯奕明）	36
5.4 两级内存分配器（韩硕）	37
5.5 多调度算法框架（涂凡琛）	37
六、 总结与展望	38
6.1 项目总结	38
6.2 未来展望	39
6.2.1 网络协议栈增强（侯奕明）	39
6.2.2 系统调用扩展	39
6.2.3 调度与同步增强	39
6.2.4 文件系统增强	40
6.3 个人收获（侯奕明）	40

一、项目概述

1.1 项目背景

操作系统是计算机系统的核心软件，负责管理硬件资源、提供程序运行环境。本课程设计要求学生基于 xv6-riscv 教学操作系统，深入理解并扩展操作系统内核的核心功能。xv6 是由 MIT 开发的教学操作系统，最初基于 x86 架构，后移植到 RISC-V 架构。xv6-riscv 包含了操作系统最基本的功能模块：系统启动、中断与异常处理、内存管理、进程管理、文件系统以及用户程序加载与执行，是学习操作系统内核实现的理想平台。

本项目选择《操作系统课程设计》项目制方案中的 **A 题”OS 内核实现”**，以 xv6-riscv 为基础平台，围绕系统调用接口、中断与异常处理、设备驱动、进程管理、内存管理、文件系统以及网络协议栈等核心模块展开，在成功复现和理解现有系统的基础上，完成了课程要求的各项功能，并在此之上进行了大量扩展创新。

1.2 项目目标

本项目的核心目标是：

1. 成功复现 xv6-riscv 的运行环境，理解其启动流程与整体架构；
2. 按照 A 题六大模块要求，逐项完成功能实现与验证；
3. 在基础功能之上进行创新扩展，包括但不限于：新增系统调用、网络协议栈、调度算法优化、内核堆分配器等；
4. 完成完整的测试验证，确保系统稳定运行；
5. 形成规范的技术文档，为后续学习者提供参考。

1.3 团队信息与分工方案

本组共三名成员，按照 A 题六大模块进行分工，各自负责核心模块的分析、实现与文档工作。

表 1: 小组成员与分工

成员	负责模块	主要工作内容
侯奕明	接口、设备与软件交互	系统调用接口 (ps/halt/lseek/-socket)、trap 路径、控制台/串口驱动、virtio-net 网卡驱动、UDP 协议栈、Shell 功能增强
涂凡琛	进程管理与处理机调度	FCFS/MLFQ/RR 调度算法、信号量机制、wakeup 哈希队列优化、调度测试程序
韩硕	系统启动、内存管理与文件管理	启动链路与特权级切换、启动时内存探测、页表与物理页分配管理、内核堆分配器、文件描述符与路径解析、文件/目录操作

此外，整个小组共同承担了以下工作：

- xv6 项目的整体框架搭建与环境配置；
- QEMU 运行验证与基础调试；
- 模块间联调与问题定位；
- 课程进度报告、结题报告和汇报材料整理。

1.4 项目完成情况概览

截至结题阶段，本组已完成的工作包括：

表 2: 项目完成情况总览

功能模块	基础功能	扩展功能	测试状态
系统启动与初始化	已完成	—	✓ 通过
中断与异常处理	已完成	—	✓ 通过
系统调用接口	已完成	新增 4 个	✓ 通过
设备驱动 (UART/virtio)	已完成	virtio-net	✓ 通过
控制台 I/O	已完成	输出原子化	✓ 通过
Shell 功能增强	已完成	PATH fallback- /halt 内置	✓ 通过
进程管理与调度	已完成	FCFS/MLFQ	✓ 通过
信号量机制	已完成	3 组测试	✓ 通过
内存管理	已完成	kmalloc 堆分配器	✓ 通过
文件管理	已完成	lseek/文件描述符管理/路径解析	✓ 通过
网络协议栈 (UDP)	新增	全协议栈	✓ 通过
wakeup 哈希队列优化	已完成	$O(N) \rightarrow O(k)$	✓ 通过

二、 技术方案设计

2.1 整体架构

本项目的技术路线是以 xv6-riscv 为基础实验平台，结合课程 A 题要求完成三步工作：复现验证现有功能、补齐缺失功能、创新扩展新功能。整体系统架构如图1所示。



图 1: 系统整体架构图

2.2 模块划分

系统从功能角度划分为以下核心模块：

- 系统启动模块：**从 M-mode 到 S-mode 的切换、内核初始化、第一个用户进程创建；
- 中断与异常处理模块：**trap 入口设置、系统调用分发、时钟中断、设备中断、缺页异常处理；
- 系统调用接口模块：**系统调用注册、参数传递、返回值处理、新增系统调用；
- 设备驱动模块：**UART 串口驱动、virtio 磁盘驱动、virtio-net 网卡驱动、PLIC 中断控制器；
- 进程管理模块：**进程生命周期、调度算法 (RR/FCFS/MLFQ)、信号量同步；

- 6. 内存管理模块：页级分配器（kalloc）、内核堆分配器（kmalloc）、页表管理；
- 7. 文件系统模块：inode 层、目录与路径解析、文件描述符管理、lseek；
- 8. 网络协议栈模块：Ethernet/ARP/IP/UDP 四层协议、socket 系统调用接口。

2.3 技术选型

表 3: 技术选型说明

技术项	选择与理由
指令集架构	RISC-V (rv64gc): 开源、简洁、现代, xv6 原生支持
模拟平台	QEMU 8.2.2 (virt 机器): 完整的 RISC-V 系统模拟, 支持 MMIO 设备
内核基座	xv6-riscv: MIT 教学内核, 代码精简 (约 8000 行), 适合学习与扩展
编译器	riscv64-unknown-elf-gcc: RISC-V 交叉编译工具链
构建系统	GNU Make + Perl (usys.pl 自动生成系统调用 stub)
网络设备	virtio-net MMIO: 相比 E1000 PCIe 在 QEMU RISC-V 上更稳定 (避免 PCIe DMA bug)
调度算法	编译期宏选择 (SCHED_RR / SCHED_FCFS / SCHED_MLFQ), 避免运行时开销
内存分配	两级架构: kalloc (4KB 页) + kmalloc (slab 小对象 + 大对象回退)

三、 模块详细实现

本章按照分工详细描述各模块的设计思路、数据结构、关键算法和核心代码实现。其中第 3.1 至 3.5 节为侯奕明负责部分（重点），第 3.6 节为涂凡琛负责部分，第 3.7 至 3.9 节为韩硕负责部分，分别对应系统启动、内存管理与文件管理三个模块。

3.1 中断与异常处理（侯奕明）

3.1.1 设计思路

中断与异常处理是操作系统的核心机制，负责管理系统调用（ecall）、时钟中断、设备中断和缺页异常。我（侯奕明）负责对该模块进行全面分析和实现，重点包括：

- 系统调用全链路：从用户态 ecall 到内核 syscall 分发再到 sret 返回的完整路径；
- 中断处理的两条路径：用户态被中断（uservec→usertrap）vs 内核态被中断（kernelvec→kerneltrap）；
- 时钟中断与调度触发点（yield）；
- 控制台 I/O 的中断驱动数据流。

3.1.2 系统调用全链路

一次系统调用的完整生命周期是理解用户态与内核态交互的核心。以 `write(1, "hello", 5)` 为例，其完整路径如下：

1. **用户态准备**：用户程序调用 `write()` → `user/usys.pl` 生成的汇编 stub 执行 `li a7, SYS_write; ecall; ret`
2. **硬件陷入**：ecall 指令触发 trap，CPU 硬件将 `scause` 设为 8（Environment call from U-mode），`sepc` 设为 ecall 指令地址，`stvec` 指向 trampoline 页中的 `uservec`
3. **上下文保存**：trampoline.S:uservec 保存全部 32 个通用寄存器到 trapframe，切换页表（用户页表 → 内核页表），跳转到 `usertrap()`
4. **Trap 分发**：trap.c:usertrap() 检查 `scause`：
 - `scause == 8`：系统调用 → 调用 `syscall()`
 - `scause == 13/15`：缺页异常 → 调用 `vmfault()`
 - `scause & 0x8000000000000000`：中断 → 调用 `devintr()`
5. **系统调用分发**：syscall.c:syscall() 从 trapframe->a7 取系统调用号，查 `syscalls[]` 表，调用对应 `sys_write()`，返回值写入 trapframe->a0

6. **返回准备**: `trap.c:prepare_return()` 关中断, 设置 `stvec=uservec` (为下次用户态 trap 做准备), 设置 `SPP=0` (让 `sret` 回到 U-mode)、`SPIE=1` (回到用户态后开中断)
7. **恢复现场**: `trampoline.S:userret` 从 `trapframe` 恢复全部寄存器, 切回用户页表, 执行 `sret`

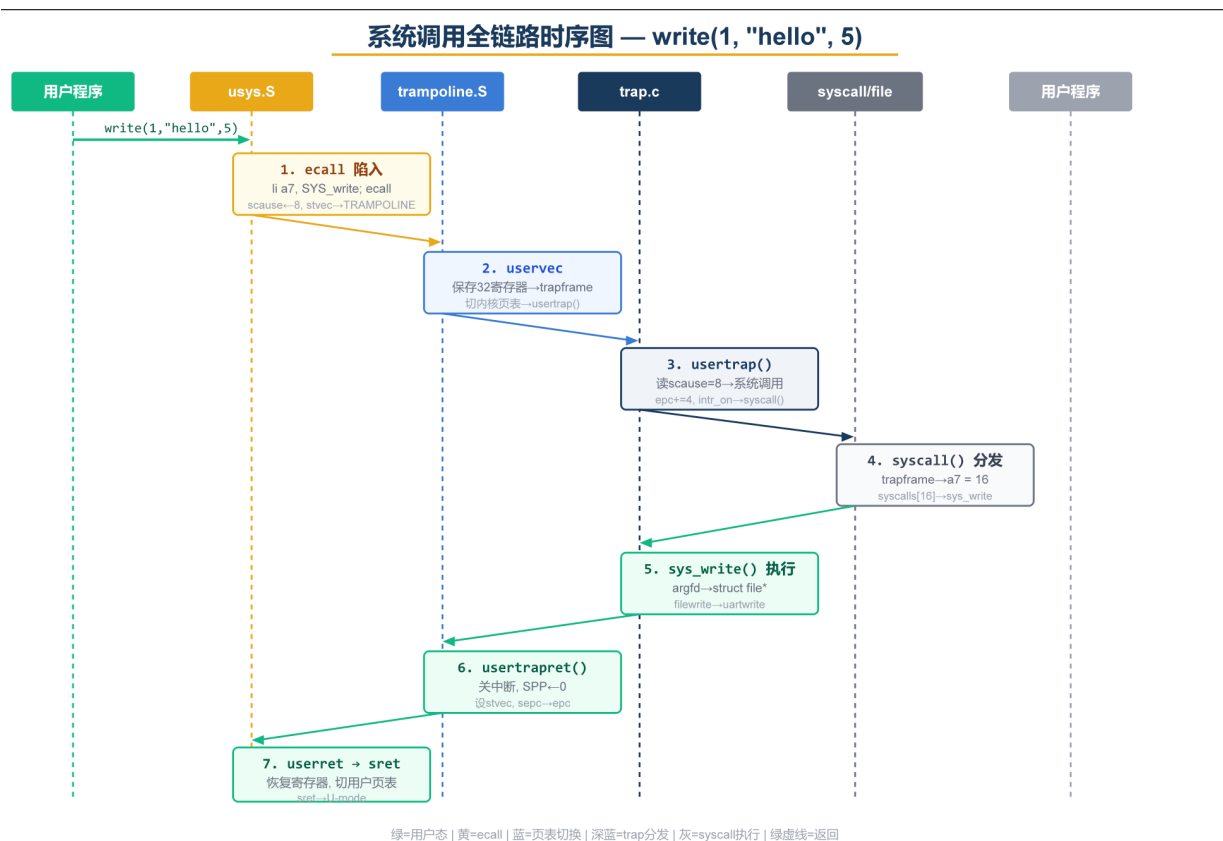


图 2: 系统调用全链路时序图

3.1.3 中断处理的两条路径

中断处理根据被中断时的特权级分为两条路径:

路径 A: 用户态被中断

用户进程运行中

→ 时钟/设备中断触发

→ `uservec` (`trampoline.S`) 保存寄存器、切内核页表

→ `usertrap()` 分发:

→ `devintr()` 判断中断类型:

· `scause=0x8000000000000005` → `clockintr()` → 返回 2

· `scause=0x8000000000000009` → `plic_claim()` → 设备处理

→ `which_dev==2` 则 `yield()` (时间片调度触发点)

- `prepare_return()` 准备返回用户态
- `userret (trampoline.S)` 恢复寄存器、`sret`
- 回到用户态继续执行

路径 B：内核态被中断

内核代码运行中

- 时钟/设备中断触发
- `kernelvec (kernelvec.S)` 保存寄存器
- `kerneltrap()` 处理：
 - `devintr()` 同样判断中断类型
 - `which_dev==2` 则 `yield()`
 - 恢复 `sepc/sstatus`
- `sret` 回到内核继续执行

关键代码位置：

- `trap.c:85` — `if (which_dev == 2) yield()` — 时间片调度的触发点
- `trap.c:187-220` — `devintr()` 的完整分流逻辑
- `trap.c:167-180` — `clockintr()`: `ticks++`、`wakeup`、预约下一次时钟中断
- `trap.c:29` vs `trap.c:112` — `stvec` 在内核态 vs 回用户态前的切换

3.1.4 时钟中断机制

Listing 1: `clockintr()` — 时钟中断处理 (`kernel/trap.c:167-180`)

```

1 void clockintr() {
2     // 递增系统时钟计数
3     ticks++;
4
5     // 唤醒所有等待时钟的进程 (如 pause())
6     wakeup(&ticks);
7
8     // 预约下一次时钟中断 (约 10ms 后, 即约 100Hz)
9     w_stimecmp(r_time() + 1000000);
10 }
```

时钟中断频率由 `start.c` 中的 `timerinit()` 设定，通过设置 `stimecmp` 寄存器预约第一次中断，之后每次 `clockintr()` 中预约下一次，形成约 100Hz 的稳定时钟中断流。时钟中断是时间片轮转调度、`pause()` 睡眠/唤醒等机制的基础。

3.2 系统调用接口实现（侯奕明）

我负责了全部新增系统调用的设计与实现，包括 `ps`（进程列表）、`halt`（关机）、`lseek`（文件定位）以及 4 个 `socket` 网络系统调用。

3.2.1 `ps` 系统调用

设计动机： xv6 原有的进程信息查看只能通过 Ctrl-P 在内核态触发 `procdump()`，用户态程序无法获取进程列表。新增 `ps` 系统调用将其暴露给用户态。

实现策略： 零新增内核逻辑——`sys_ps()` 直接复用 `proc.c` 中已有的 `procdump()`。

修改文件清单：

表 4: `ps` 系统调用修改文件

文件	修改内容
<code>kernel/syscall.h</code>	新增 <code>#define SYS_ps 22</code>
<code>kernel/syscall.c</code>	新增声明 + <code>syscalls[]</code> 表中注册 <code>[SYS_ps]</code> <code>sys_ps</code>
<code>kernel/sysproc.c</code>	新增 <code>sys_ps()</code> 函数，直接调用 <code>procdump()</code>
<code>user/usys.pl</code>	新增 <code>entry("ps");</code> —自动生成用户态汇编 stub
<code>user/user.h</code>	新增 <code>int ps(void);</code> —用户态函数声明
<code>user/ps.c</code>	新建用户程序，调用 <code>ps()</code> 后 <code>exit(0)</code>

调用路径：

Listing 2: `sys_ps()` 实现（`kernel/sysproc.c`）

```

1 uint64 sys_ps(void) {
2     procdump();    // 直接复用内核已有的 procdump()
3     return 0;
4 }
```

`ps.c`: `ps()` → `usys.S stub (li a7, SYS_ps; ecall; ret)`

→ `trap.c`: `usertrap()` 捕获 `ecall (scause=8)`

→ `syscall.c`: `syscall()` 查 `syscalls[]` 表

→ `sysproc.c`: `sys_ps()` → `procdump()`

`procdump()` 遍历 `proc[NPROC]`，打印每个非 `UNUSED` 进程的 `pid`、状态（`unused/used/sleep-/runble/run/zombie`）、进程名。

3.2.2 halt 关机系统调用

设计动机：xv6 原始实现没有关机机制，用户只能通过关闭 QEMU 窗口退出。新增 `halt` 系统调用允许从 shell 中正常关机。

实现策略：`halt` 设计为 shell 内建命令（类似 `cd`），不经过 `fork/exec`，直接在 shell 进程中执行。同时也保留 `user/halt.c` 作为独立程序的备用路径。

调试历程中的关键问题与解决：

1. 问题 1：SBI ecall 导致内核 panic

现象：`scause=0x9 sepc=0x800039ec panic: kerneltrap`

原因：`start.c` 中 `w_medeleg(0xffff)` 将所有异常（包括 bit 9: Environment call from S-mode）委托给 S-mode。内核执行 `ecall` 时，`trap` 被 `kerneltrap()` 捕获，OpenSBI 收不到请求。

修复：`w_medeleg(0xffff & ~(1L << 9))` — 排除 bit 9，让 S-mode `ecall` 到达 M-mode（OpenSBI）。

2. 问题 2：SBI ecall 发出后 QEMU 不退出

现象：`halt` 命令后 QEMU 挂住不退出。

原因：xv6 使用 `-bios none -kernel` 启动，没有 OpenSBI 固件。内核从 M-mode 切换到 S-mode 后未设置 `mtvec`，`ecall` 到 M-mode 时跳转到地址 0 处挂死。

修复：放弃 SBI `ecall` 方案，改用 **QEMU test device**（SiFive test device，位于物理地址 `0x100000`）直接退出 QEMU：

Listing 3: `halt` 关机最终实现（`kernel/sysproc.c`）

```

1 // memlayout.h 中定义
2 #define TEST_DEVICE 0x100000L
3
4 // vm.c 中添加 identity mapping
5 kvmmap(kpgtbl, TEST_DEVICE, TEST_DEVICE, PGSIZE, PTE_R | PTE_W);
6
7 // sysproc.c 最终实现
8 uint64 sys_halt(void) {
9     volatile uint32 *test_dev = (volatile uint32 *)TEST_DEVICE;
10    *test_dev = 0x5555; // 写 QEMU test device → QEMU 退出
11    return 0;
12 }
```

工作原理：QEMU virt 机器内置 SiFive test device (MMIO 地址 0x100000)，向其写入任意值触发 `qemu_system_shutdown_request_with_code()` 退出。内核页表对该地址做 identity mapping 后，S-mode 内核即可直接访问。

修改文件：9 个文件（1 新建 + 8 修改），涉及系统调用注册、用户态 stub、shell 内建命令检测、内核页表映射、Makefile 编译链等。

3.2.3 lseek 文件定位系统调用

设计动机：xv6 原始实现的 `read/write` 只能顺序读写文件，不支持随机访问。新增 `lseek` 系统调用允许用户程序重新定位文件读写偏移量。

三种定位模式：

表 5: `lseek` 三种 whence 模式

模式	常量	基准点	示例
<code>SEEK_SET</code>	0	文件开头	<code>lseek(fd, 10, SEEK_SET)</code> → <code>f->off = 10</code>
<code>SEEK_CUR</code>	1	当前位置	<code>lseek(fd, -5, SEEK_CUR)</code> → <code>f->off -= 5</code>
<code>SEEK_END</code>	2	文件末尾	<code>lseek(fd, -6, SEEK_END)</code> → <code>f->off = size - 6</code>

Listing 4: `sys_lseek()` 核心实现 (kernel/sysfile.c)

```

1  uint64 sys_lseek(void) {
2      struct file *f;
3      int offset, whence;
4
5      if (argfd(0, 0, &f) < 0)    return -1;
6      argint(1, &offset);
7      argint(2, &whence);
8
9      if (f->type != FD_INODE)    // 只对普通文件支持 lseek
10         return -1;
11
12     ilock(f->ip);
13
14     switch (whence) {
15     case SEEK_SET:
16         if (offset < 0) goto bad;

```

```
17     f->off = offset;
18     break;
19 case SEEK_CUR:
20     if (offset < 0 && f->off < (uint)(-offset)) goto bad;
21     f->off += offset;
22     break;
23 case SEEK_END:
24     if (offset < 0 && f->ip->size < (uint)(-offset)) goto bad;
25     f->off = f->ip->size + offset;
26     break;
27 default:
28     goto bad;
29 }
30
31 iunlock(f->ip);
32 return f->off;    // 返回新的偏移量
33
34 bad:
35     iunlock(f->ip);
36     return -1;
37 }
```

设计要点：

- 只作用于普通文件（FD_INODE），管道和设备返回 -1；
- 三种模式均包含下溢边界保护；
- inode sleeplock（ilock/iunlock）保护关键区；
- 成功返回新偏移量，与 POSIX 行为一致；
- read/write 会自动从 f->off 处读写并推进偏移量，无需额外联动。

典型用法：lseek(fd, 0, SEEK_END) 返回文件当前大小，是最常用的文件大小获取技巧。

3.2.4 Socket 网络系统调用

为支撑 UDP 网络协议栈，新增了 4 个 socket 系统调用：

表 6: 新增 Socket 系统调用

系统调用	编号	函数签名	说明
socket	23	int socket(domain, type, protocol)	创建 UDP socket
bind	24	int bind(sockfd, ip, port)	绑定本地端口
sendto	25	int sendto(sockfd, buf, len, dst_ip, dst_port)	发送 UDP 包
recvfrom	26	int recvfrom(sockfd, buf, maxlen, src_ip, src_port)	接收 UDP 包

文件类型扩展：在 file.h 中新增 FD_SOCK 类型，socket 复用 xv6 的文件描述符系统。每个 socket 维护独立的接收环形缓冲区（4096 字节），通过自旋锁保护并发访问。

3.3 设备驱动与 I/O (侯奕明)

3.3.1 控制台输出原子化

问题描述：多进程并发 printf 时，由于 xv6 的 printf 每字符调一次 write()，且内核 consolewrite() 无锁保护，导致两个进程的输出逐字符交错，严重时可读性几乎为零。

根因分析：

层次	问题	后果
用户态	printf 每字符调一次 write()	一次 printf = N 次系统调用
内核态	consolewrite() 无锁保护	N 次 write 之间调度器可切进程

修复方案：两层保护协同：

- 1. **内核层：**console.c 中 cons 结构体新增 sleeplock writelock，consolewrite() 首尾加 acquiresleep/releasesleep
- 2. **用户态层：**重写 printf.c 的 vprintf()，引入动态缓冲机制：

- 默认使用 256 字节栈缓冲区;
- 超过容量时自动 `malloc` 双倍扩容;
- 格式化完成后调用一次 `write(fd, buf, len)` —— 整个 `printf` 一次 `write`。

Listing 5: 动态缓冲核心数据结构 (user/printf.c)

```

1 struct bufstate {
2     char *buf;          // 当前缓冲区指针
3     int len;            // 已写入字节数
4     int cap;            // 缓冲区总容量
5     int on_heap;        // buf 是否来自 malloc (需要 free)
6 };
7
8 static void bufputc(struct bufstate *bs, char c) {
9     if (bs->len >= bs->cap) {
10        // 容量不足 → malloc 双倍扩容
11        int newcap = bs->cap * 2;
12        char *newbuf = malloc(newcap);
13        if (newbuf == 0) {
14            // malloc 失败: flush 已有内容, 重置
15            write(1, bs->buf, bs->len);
16            bs->len = 0;
17            return;
18        }
19        // 拷贝旧内容到新缓冲区
20        for (int j = 0; j < bs->len; j++)
21            newbuf[j] = bs->buf[j];
22        if (bs->on_heap) free(bs->buf);
23        bs->buf = newbuf;
24        bs->cap = newcap;
25        bs->on_heap = 1;
26    }
27    bs->buf[bs->len++] = c;
28 }

```

协同工作流程:

【用户态】`printf(fmt, ...)`

→ `vprintf()` 格式化到 `bufstate` (栈缓冲或 `malloc` 扩容)

→ 格式化完成后, 调用一次 `write(fd, buf, len)` (整个 `printf` 仅一次 `write`)

【内核态】`consolewrite()`

→ `acquiresleep(&cons.writelock)` (拿锁, 后续 write 排队等待)
→ 批量 `uartwrite()` 输出全部字符
→ `releasesleep(&cons.writelock)` (放锁, 下一个 write 可进入)

为什么用 `sleep lock` 而非 `spinlock`? `uartwrite()` 内部可能因为 UART 发送缓冲区满而调用 `sleep()`。xv6 规定持 `spinlock` 时不能 `sleep` (会触发 `panic: sched locks`), 所以必须用允许 `sleep` 的 `sleeplock`。

3.3.2 virtio-net MMIO 网卡驱动

为支撑 UDP 网络协议栈, 实现了基于 virtio-net MMIO 的网卡驱动。
为什么选择 virtio-net 而非 E1000 ?

方面	E1000 (PCIe)	virtio-net (MMIO)
传输方式	PCIe DMA (descriptor rings)	共享内存 virtqueue
依赖	PCI ECAM 枚举 + MSI-X/INTx	直接 MMIO 访问
QEMU RISC-V 支持	PCIe DMA 有 bug (DD 位永远不置)	完全可用
复杂度	高 (PCI BAR 映射、MSI-X 配置)	低 (kvmmap 一页即可)

经过实际调试发现, QEMU 8.2.2 的 RISC-V virt 机器上 GPEX PCIe 主机桥的 DMA 写入存在兼容性问题, E1000 发送描述符的 DD 位永远不被写入。virtio-net 使用共享内存通信, 完全避免了 PCIe DMA 问题。

virtio MMIO 地址布局 (QEMU 8.2.2, `force-legacy=false`):

地址	Bus Slot	IRQ	用途
0x10001000	bus.0	1	virtio-blk (磁盘, VIRTIO0)
0x10002000	bus.1	2	virtio-net (网卡, VIRTIO1)
0x10003000	bus.2	3	(未使用)
...	...	...	...
0x10008000	bus.7	8	(未使用)

设备初始化流程:

1. 读取 MMIO 寄存器验证设备 (Magic Value: 0x74726976 "virt"、Device ID: 1 = VIRTIO_ID_NET)
2. 走设备状态机: ACKNOWLEDGE → DRIVER → FEATURES_OK → DRIVER_OK
3. 初始化 RX virtqueue (Queue 0): 8 个 buffer 预填充为 WRITE
4. 初始化 TX virtqueue (Queue 1): 所有 descriptor 初始为空闲
5. 从 VIRTIO_MMIO_CONFIG 读取 6 字节 MAC 地址

数据收发流程:

发送 (virtio_net_transmit):

1. 找空闲 TX descriptor
2. 将数据拷贝到 tx_buf, 前面预留 virtio_net_hdr (10 字节)
3. 设置 descriptor: addr, len, flags=0 (device reads)
4. 添加到 avail ring, 写 VIRTIO_MMIO_QUEUE_NOTIFY 通知设备
5. 轮询等待传输完成

接收 (virtio_net_receive):

1. 检查 used ring 是否有新条目
2. 读取 pkt_len, 跳过 virtio_net_hdr
3. 拷贝以太网帧到 buf
4. 回收 descriptor 到 avail ring

3.4 Shell 功能增强 (侯奕明)

3.4.1 Shell PATH Fallback

问题描述: xv6 的 exec() 只在当前工作目录查找可执行文件。当用户 cd 进入子目录后, 所有系统程序 (ls、cat、echo 等) 都在根目录 / 下, 导致所有命令报 exec xxx failed。

解决方案: 修改 user/sh.c 中 runcmd() 的 EXEC 分支, 增加 PATH fallback 机制:

Listing 6: Shell PATH fallback (user/sh.c:runcmd())

```

1 case EXEC:
2     ecmd = (struct execcmd *)cmd;
3     if (ecmd->argv[0] == 0) exit(1);

```

```

4     exec(ecmd->argv[0], ecmd->argv);           // 第一次尝试: 原始路径
5     // PATH fallback
6     if (ecmd->argv[0][0] != '/') {             // 非绝对路径
7         char fullpath[128];
8         char *prog = ecmd->argv[0];
9         if (prog[0] == '.' && prog[1] == '/')    // 去掉 "./" 前缀
10            prog += 2;
11        fullpath[0] = '/';
12        strcpy(fullpath + 1, prog);             // 拼接 "/程序名"
13        exec(fullpath, ecmd->argv);             // 第二次尝试: 根目录
14    }
15    fprintf(2, "exec %s failed\n", ecmd->argv[0]);
16    break;

```

逻辑说明:

步骤	说明
第 1 次 exec	用原始参数执行 (保持向后兼容)
检查路径	如果以 / 开头则跳过 fallback (已是绝对路径)
去掉 ./	如果命令以 ./ 开头, 去除此前缀
拼接路径	在程序名前加上 /, 如 ls → /ls
第 2 次 exec	用绝对路径重试执行

特点: 向后兼容 (不影响绝对路径和当前目录程序)、零性能开销 (仅在首次 exec 失败时触发)、只 fallback 到 / 根目录确保安全性。

3.4.2 halt 内建命令

halt 实现为 shell 内建命令, 检测逻辑放在 main() 的 while 循环中 (cd 检测之前):

Listing 7: halt 内建命令检测 (user/sh.c)

```

1  if (cmd[0] == 'h' && cmd[1] == 'a' && cmd[2] == 'l' && cmd[3] == 't'
    &&
2      (cmd[4] == '\n' || cmd[4] == ' ' || cmd[4] == '\0')) {
3      shutdown_animation();
4      halt();
5  }

```

halt 不经过 fork 的原因是: 即使文件系统出问题导致 exec 失败, 用户仍能正常关机。

3.5 网络协议栈实现（侯奕明）

3.5.1 总体架构

在 xv6-riscv 中实现了基于 virtio-net MMIO 网卡驱动的整体 UDP 网络协议栈，包含 Ethernet、ARP、IP、UDP 四层协议以及 socket 系统调用接口。



图 3: UDP 网络协议栈分层架构

3.5.2 Ethernet / ARP / IP / UDP 协议实现

报文格式:

Ethernet(14) | IP(20) | UDP(8) | Payload(0-1472)

加上 virtio-net header (10 字节), 总帧不超过 2048 字节 (NET_BUF_SZ)。

ARP 地址解析:

ARP 表设计为 `arp_table[16]`, 以 (IP, MAC) 键值对存储。发送流程:

1. `arp_lookup(dst_ip)` 查找目标 MAC
2. 未命中 → `arp_send_request()` 广播 ARP 请求到 FF:FF:FF:FF:FF:FF

3. 轮询 `net_rx_loop()` 等待 ARP 回复 (最多 2000 次, 约 2 秒)
4. 收到 ARP reply → `arp_add()` 记录到表
5. 超时返回 -1

IP 校验和:

采用 16-bit 反码求和 (RFC 791), 仅计算 IP 头校验和, UDP 校验和设为 0 (禁用)。

3.5.3 Socket 层设计

Socket 结构体:

Listing 8: Socket 结构体 (kernel/sysnet.c)

```

1  #define NSOCK 16  // 最多同时 16 个 socket
2
3  struct socket {
4      int    used;          // 是否已分配
5      int    fd;            // 文件描述符编号
6      uint32 raddr;         // 远端 IP
7      uint16 lport;         // 本地端口
8      uint16 rport;         // 远端端口
9      char   rbuf[4096];    // 接收环形缓冲区
10     int    rhead, rtail;   // 读/写指针
11     int    rcount;         // 缓冲区中字节数
12     struct spinlock lock;
13 };

```

sendto 完整路径:

```

udptest.c: sendto()
→ sysnet.c: sys_sendto()
→ copyin() 从用户空间拷贝数据
→ net.c: udp_send(dst_ip, dport, sport, buf, len)
→ arp_lookup(dst_ip)
    · 命中 → 直接用已知 MAC
    · 未命中 → arp_send_request() 广播 ARP
        → net_rx_loop() 轮询等待 ARP 回复
        → arp_recv() 处理 ARP 回复, 记录 MAC
→ 构建完整报文 [Eth(14) + IP(20) + UDP(8) + payload]
→ virtio_net_transmit() 发送到 TX virtqueue

```

3.5.4 QEMU 网络配置

Listing 9: Makefile QEMU 网络参数

```

1 QEMUOPTS += -device virtio-net-device,netdev=net0,bus=virtio-mmio-bus
   .1 \
2         -netdev user,id=net0

```

关键点：

- `virtio-net-device`：使用 MMIO 而非 PCI 的 virtio 网卡
- `bus=virtio-mmio-bus.1`：必须显式指定总线槽位，否则 QEMU 8.2.2 自动分配不可靠
- `-netdev user`：SLIRP user-mode 网络，提供 NAT 服务
- `-global virtio-mmio.force-legacy=false`：使用新地址布局

网络拓扑：

xv6 Guest (10.0.2.15) ↔ QEMU SLIRP Gateway (10.0.2.2) ↔ Host OS

3.5.5 调试历程总结

1. **E1000 PCIe DMA 故障**：调试确认 PCI 枚举和 BAR 映射正确后，发现 DD 位始终为 0。根因为 QEMU 8.2.2 RISC-V virt 的 GPEX PCIe 主机桥 DMA 写回 bug。解决方案：放弃 E1000，改用 virtio-net MMIO。
2. **virtio MMIO 地址定位**：初始设 VIRTIO1 为 0x10001200，实际应为 0x10002000。通过 dumpdtb 导出设备树解析获得正确地址。
3. **总线槽位显式指定**：不加 `bus=` 参数时 QEMU 自动分配位置不可靠，必须显式指定。

3.6 进程管理与调度（涂凡琛）

本部分由涂凡琛负责，主要包括以下工作。

3.6.1 FCFS 与 MLFQ 调度算法实现

在 xv6 原有 RR（时间片轮转）调度器基础上，新增了 FCFS（先来先服务）和 MLFQ（多级反馈队列）两种调度算法。

进程结构扩展（`kernel/proc.h`）：


```

1 uint64 ctime;           // 进程创建时间 (ticks) — 用于 FCFS
2 int queue_level;       // MLFQ 队列级别 (0=最高, 1=中, 2=最低)
3 int timeslice_used;    // 本时间片已用 tick 数
4 uint64 last_sched;     // 上次被调度的时间
5 int priority;          // 进程优先级

```

调度算法选择 (编译期宏, kernel/param.h):

```

1 #define SCHED_RR      0    // 时间片轮转 (默认)
2 #define SCHED_FCFS    1    // 先来先服务
3 #define SCHED_MLFQ    2    // 多级反馈队列
4 #define SCHED_ALGORITHM SCHED_RR

```

MLFQ 调度参数:

- 3 级队列: Q0 (5ms)、Q1 (10ms)、Q2 (20ms)
- 时间片用完降级
- 每 200 ticks 优先级提升, 防止饥饿

3.6.2 信号量机制

在 xv6-riscv 上实现了完整的信号量机制, 基于内核的 `sleep()`/`wakeup()` 机制:

- **数据结构:** `struct semaphore` 包含自旋锁、计数值、等待队列
- **P/V 操作:** `sem_wait()` (P 操作) 递减并可能阻塞; `sem_post()` (V 操作) 递增并唤醒等待者
- **系统调用:** `sem_open/sem_wait/sem_post/sem_get/sem_close` (编号 24-28)
- **测试程序:** `semtest1` (基本 P/V)、`semtest2` (互斥锁, 4 进程 800 次操作)、`semtest3` (生产者-消费者, 2 生产者 +2 消费者, 20 个项目)

3.6.3 wakeup 哈希队列优化

将 `wakeup(chan)` 从原来的全进程表扫描 ($O(NPROC)$) 优化为按 `channel` 定位等待桶后局部遍历 ($O(k)$):

- 引入 64 个等待桶 (`NWCHAN 64`), 每个桶有独立的 `spinlock` 和进程链表头
- `waitbucket_for(chan)` 通过哈希定位桶
- `sleep()` 中插入桶链表, `wakeup()` 中遍历桶链表
- 进程结构中新增 `wnext/wprev/wbucket` 字段

3.7 系统启动 (韩硕)

本节主要介绍系统从 QEMU 启动到进入 xv6 内核主函数，再到拉起第一个用户态 shell 的完整过程。相比原始 xv6 的直接阅读，本报告更强调“启动链路中每一步做了什么、为什么必须这么做、以及当前实现已经验证到什么程度”。

3.7.1 启动链路

当前系统已经实现 RISC-V 平台下从 QEMU 启动到进入 shell 的完整启动链路，核心路径为：

_entry → start() → main() → userinit() → init → sh

启动入口与加载位置：

- QEMU 通过 `-kernel` 将内核镜像加载到 `0x80000000`
- `kernel.ld` 将最早入口 `_entry` 放在该地址
- CPU 上电后从 `kernel/entry.S` 开始执行

3.7.2 关键初始化步骤

早期栈与 C 运行环境准备：

- `entry.S` 为每个 hart 计算独立启动栈 `stack0`
- 在跳转到 `start()` 之前就完成 `sp` 初始化
- 启动参数中的 DTB 指针也在该阶段从 `a1` 保存下来，供后续内存探测使用

M 态到 S 态切换：

- `start()` 中设置 `mstatus.MPP = S`
- 将 `mepc` 指向 `main`
- 完成中断/异常委托、PMP 配置、定时器初始化和 `tp = hartid`
- 最终通过 `mret` 切换到 S 态并进入内核主函数

3.7.3 课程要求对应与实现结果

Trap 向量与启动结果：

- 系统通过 `stvec` 安装 RISC-V trap 向量，等价完成课程要求中的中断向量初始化
- `main()` 中继续完成控制台、内存、页表、进程、设备与文件系统初始化

- 最终创建第一个用户进程并进入 shell，说明启动链路已经稳定闭环

从课程要求角度看，当前系统启动模块已经覆盖以下关键指标：

- 能在模拟器中稳定启动并进入内核主函数
- 已完成从 M 态到 S 态的切换
- 已初始化 trap 向量
- 已为每个 CPU 设置启动栈指针，为 C 语言运行环境做好准备
- 启动后能继续完成设备、文件系统和首个用户进程初始化，最终进入交互式 shell

这说明系统启动并不是“只能跑起来”，而是已经具备了完整、可复现、可继续扩展的内核启动骨架。

3.8 内存管理（韩硕）

本节主要介绍物理内存、虚拟内存以及内核动态分配三层能力。当前实现不再仅依赖 xv6 默认的固定物理内存上界，而是补上了运行时 RAM 探测，并将其接入页分配器、页表映射和共享内存合法性判断。

3.8.1 物理页框分配与启动时内存探测

当前内存管理由“物理页框分配 + 虚拟内存管理 + 内核堆分配器”三部分组成。

物理页框分配器：

- 底层 kalloc/kfree 采用基于空闲链表的简单页分配器
- 每个空闲页使用 struct run 链入 kmem.freelist
- 分配与回收粒度统一为 4KB 页框

启动时物理内存探测：

- 新增启动时 DTB 解析能力，读取 /memory/reg 获取 RAM 起止范围
- 运行时维护 phys_ram_start、phys_ram_end、phys_ram_detected
- kinit() 将内核镜像结尾 end 到 phys_ram_end 之间的页加入空闲链表
- kfree() 合法性检查、内核直接映射和共享内存地址校验也统一使用运行时探测结果

3.8.2 虚拟内存与按需分页

虚拟内存管理:

- 页表创建 (kvmmake/uvmmcreate/proc_pagetable)
- 页表映射 (walk/mappages/kvmmmap)
- 地址解除映射 (uvmunmap/uvmmdealloc/uvmmfree)
- 用户地址空间布局 (代码段、数据段、堆、栈、TRAPFRAME、TRAMPOLINE)
- 缺页处理与 lazy sbrk, 支持首次访问时再分配物理页

用户地址空间布局:

- 低地址部分依次是代码段、数据段与 bss
- 中间通过 sbrk 扩展堆空间
- 高地址部分保留用户栈、TRAPFRAME 和 TRAMPOLINE
- exec() 负责把 ELF 中的各段装入页表, 并为用户栈压入参数

当前系统已经支持基于 lazy sbrk 的按需分页: 调用 sbrk 时先只增加逻辑地址空间大小, 真正访问未映射页时再通过缺页异常进入 vmfault(), 由内核分配物理页并建立映射。这种实现方式虽然不包含 swap 或页面置换, 但已经满足课程要求中的 Demand Paging 基本能力。

3.8.3 内核堆分配器与课程指标

内核堆分配器:

- 在 xv6 原有页级分配器之上实现 kmalloc/kmfree 两级架构
- 采用 7 个 size-class slab 档位 (32B/64B/128B/256B/512B/1024B/2048B)
- 2048B 以上对象自动回退到整页分配
- 空 slab 页会自动归还给 kfree()
- struct pipe 和 struct file 已从整页分配改为 kmalloc 精确分配

接口:

```
1 void  kheapinit(void);           // 初始化堆分配器
2 void *kmalloc(uint n);          // 分配 n 字节
3 void  kmfree(void *p);          // 释放对象
```

课程指标对应：

- 已支持远超 4MB 的物理内存管理，实测在 QEMU -m 128M 和 -m 256M 下均能正确识别
- 页大小为 4KB
- 已支持基于 lazy sbrk 的按需分页
- 普通页分配路径支持释放后重用；若将扩展共享内存路径一并纳入，则“绝对无泄漏”仍需保留严格性说明

综合来看，当前内存管理已经形成“运行时物理内存探测 + 4KB 页框分配 + Sv39 页表管理 + slab 堆分配器”的完整层次，既覆盖了课程要求，也为后续继续实现更复杂的虚拟内存机制保留了扩展空间。

3.9 文件管理（韩硕）

本节主要介绍文件描述符表、文件/目录基本操作、路径解析和 lseek 支持。这里采用“从进程视角到文件系统视角”的写法，便于说明一个用户态 fd 最终如何映射到底层 inode 和磁盘数据块。

3.9.1 文件描述符表管理

文件管理部分不仅包括文件描述符表，还包括文件/目录操作、路径解析与 lseek 支持。

文件描述符表管理：

- 每进程独立文件描述符表 ofile[NOFILE]
- fd 分配、查找、复制 (dup)、关闭 (close)
- fork 后文件描述符继承
- exit 时自动关闭全部打开文件
- 基于引用计数的底层 struct file 生命周期管理

实现意义：

- 用户态看到的是整数 fd，内核态实际管理的是 struct file
- dup() 共享同一个打开文件对象，因此会共享文件偏移
- fork() 后父子进程继承同一组打开文件引用
- exit() 时自动关闭全部 fd，避免进程级资源泄漏

3.9.2 文件与目录基本操作

文件与目录基本操作：

- 普通文件创建通过 `open(path, O_CREATE)` 与 `create()` 完成
- 目录创建通过 `mkdir()` 与 `create(T_DIR)` 完成
- 删除通过 `unlink()` 删除目录项并递减 inode 链接计数
- 打开/关闭通过 `namei()`、`filealloc()`、`fdalloc()`、`fileclose()` 完成
- 读写通过 `fileread()/filewrite()` 调用 inode 层 `readi()/writei()` 实现

分层理解：

- `sysfile.c` 负责系统调用参数、权限与事务边界
- `file.c` 负责打开文件对象、偏移和引用计数
- `fs.c` 负责 inode、目录项、路径遍历和实际数据读写

3.9.3 路径解析与 seek 支持

路径解析与 seek：

- `namei()` / `nameiparent()` 基于 `namex()` 实现绝对路径和相对路径解析
- `skipelem()` 负责路径分段，`dirlookup()` 负责逐层目录查找
- 已补齐 `lseek`，支持 `SEEK_SET`、`SEEK_CUR`、`SEEK_END`
- 当前文件系统已满足至少 128 个文件、单文件大于 64KB、目录层级至少 3 层嵌套等课程指标

`lseek` 的作用：

- `lseek()` 本身不读写数据，只修改 `struct file` 中的当前偏移 `off`
- 后续 `read()` 和 `write()` 会从新的偏移开始工作
- 当前实现支持普通文件的偏移调整，不支持目录、管道和设备文件的随机访问

3.9.4 文件系统整体总结

从整体结构看，当前 xv6 的文件系统可以理解为一个分层的简化类 Unix inode 文件系统。最底层是 VirtIO 块设备与缓冲缓存层 `bio.c`，负责把磁盘块读入内存并缓存复用；往上日志层 `log.c`，负责保证多块更新的崩溃一致性；再往上是 `fs.c` 的 inode、目录项和路径解析逻辑，以及 `file.c` 的打开文件对象管理；最上层才是 `sysfile.c` 向用户态提供的 `open/read/write/close/mkdir/unlink/chdir` 等系统调用接口。

为什么需要日志层：

- 一次文件系统操作通常不会只改一个块，而是会同时修改 inode、位图块、目录块甚至数据块
- 如果系统在“部分块已经写盘、部分块还没写盘”时崩溃，磁盘状态就可能出现不一致
- `log.c` 的目标，就是把“涉及多个块的更新”组织成一次事务，要么整体生效，要么重启后通过恢复机制把它补完整

日志采用的技术路线：

- xv6 使用的是块级 **physical redo log**
- 日志区在磁盘上由一个日志头块和若干日志数据块组成
- 日志头记录当前事务包含多少个块，以及这些块最终要写回到哪些 home block
- 每个日志数据块保存的是“修改后的完整块内容”，而不是更细粒度的字段差异

日志在内存中的状态管理：

- `struct log` 是全局单实例，记录日志区起点、设备号、活动系统调用数量和当前日志头
- `outstanding` 表示当前有多少文件系统系统调用正在事务范围内执行
- `committing` 表示是否正在提交，避免新的修改和提交流程相互交错
- `lh` 是内存中的日志头，保存本次事务涉及的目标块号数组

事务边界：`begin_op()` 与 `end_op()`：

- 文件系统相关系统调用开始时调用 `begin_op()`，结束时调用 `end_op()`
- `begin_op()` 会检查当前是否正在提交，或者剩余日志空间是否足够；若不够则睡眠等待

- 这样做的目的是为并发中的系统调用预留日志容量，避免多个操作一起把有限的日志块耗尽
- `end_op()` 会减少 `outstanding`；如果这是最后一个活动操作，就触发真正的 `commit()`

修改记录：log_write():

- 上层修改某个缓存块后，不直接 `bwrite()` 到最终磁盘位置，而是调用 `log_write()`
- `log_write()` 只把“这个块号属于当前事务”记录到日志头中，并通过 `bpin()` 暂时固定该缓存块
- 如果同一个块在一次事务中被多次修改，日志层只记录一次块号，这就是 **log absorption**
- 真正把数据写到日志区，是在后续 `commit()` 时统一完成的

提交流程：commit():

- 第一步 `write_log()`：把缓存中的修改块内容复制到磁盘日志区的数据块
- 第二步 `write_head()`：把内存中的日志头写到磁盘日志头块，这一步才是真正的提交点
- 第三步 `install_trans()`：把日志区中的块逐个拷贝回它们各自的真实 home block
- 第四步把 `log.lh.n` 清零，再次 `write_head()`，表示本次事务已经完成并擦除日志

恢复机制：

- 文件系统初始化时，`initlog()` 会调用 `recover_from_log()`
- 该函数先读磁盘日志头；如果发现 `n > 0`，说明上次崩溃前已经有一个已提交事务写入了日志头
- 此时通过 `install_trans(1)` 把日志区内容重新安装回真实块，再清空日志头
- 这保证了系统即使在提交后、安装回 home block 之前突然崩溃，重启后仍能自动补做完整写回

日志层在整体文件系统中的意义：

- `bio.c` 解决“块怎么缓存、怎么同步访问”
- `log.c` 解决“多个块修改怎么作为一个事务安全提交”
- `fs.c` 才在这两个基础之上实现 inode、目录、位图和文件内容管理

因此，当前文件管理已经不仅具备路径解析、打开文件对象管理、文件描述符映射、文件/目录操作和偏移控制能力，还具备了由 `log.c` 提供的崩溃一致性保障。也就是说，xv6 文件系统不是简单“把块直接写盘”，而是通过日志事务机制确保多块更新在异常重启后仍能恢复到一致状态。

四、系统运行与功能测试

4.1 基础系统运行验证

4.1.1 系统启动

xv6 在 QEMU 环境中稳定启动，成功进入 shell：

Listing 10: xv6 启动日志

```
1 xv6 kernel is booting
2
3 memory: [0x80000000, 0x90000000) size=0x10000000 (dtb)
4 kmalloc_test: ok
5 [boot animation]
6 $
```

4.1.2 ps 系统调用测试

Listing 11: ps 命令运行结果

```
1 $ ps
2 pid=1 state=runble name=init
3 pid=2 state=sleep name=sh
4 pid=3 state=runble name=ps
```

ps 命令成功从用户态打印所有进程的 pid、状态和名称，验证了系统调用全链路正确性。

4.1.3 halt 关机测试

Listing 12: halt 关机测试

```
1 $ halt
2 [shutdown animation]
3 [QEMU 正常退出，返回宿主终端]
```

4.1.4 lseek 测试

Listing 13: lseektest 运行结果

```
1 $ lseektest
2 lseektest: start
3 lseektest: Test 1 — SEEK_SET
```

```
4 lseektest: read at offset 10: "ABCDEF"
5 lseektest: Test 1 — PASS
6 lseektest: Test 2 — SEEK_CUR
7 lseektest: read at offset 10: "ABCDEF"
8 lseektest: Test 2 — PASS
9 lseektest: Test 3 — SEEK_END
10 lseektest: read at offset 10: "ABCDEF"
11 lseektest: Test 3 — PASS
12 lseektest: Test 4 — file size via lseek
13 lseektest: file size = 16
14 lseektest: Test 4 — PASS
15 lseektest: ALL TESTS PASSED
```

覆盖了 SEEK_SET、SEEK_CUR、SEEK_END 三种定位模式及文件大小获取功能。

4.2 控制台输出原子化测试

4.2.1 修复前 vs 修复后

修复前（字符级交错）：

```
1 pipetest: papiprente sentte 'shtell:o c'h
2 ildp ipmetsg est0: pa: rhent seellon
```

修复后（每行独立）：

```
1 pipetest: parent sent 'hello'
2 pipetest: child msg 0: hello
3 pipetest: parent sent 'xv6'
4 pipetest: child msg 1: xv6
```

4.3 管道通信测试

Listing 14: pipetest 完整运行结果

```
1 $ pipetest
2 pipetest: start
3 pipetest: Test 1 -- ping-pong
4 pipetest: child received: ping
5 pipetest: parent received: pong
6 pipetest: Test 1 -- PASS
7 pipetest: Test 2 -- large data (768 bytes)
```

```
8 pipetest: parent wrote 768 bytes
9 pipetest: child read total 768 bytes
10 pipetest: Test 2 -- PASS
11 pipetest: Test 3 -- multiple messages
12 pipetest: parent sent 'hello'
13 pipetest: child msg 0: hello
14 pipetest: parent sent 'xv6'
15 pipetest: child msg 1: xv6
16 pipetest: parent sent 'pipe'
17 pipetest: child msg 2: pipe
18 pipetest: parent sent 'test'
19 pipetest: child msg 3: test
20 pipetest: parent sent 'world'
21 pipetest: child msg 4: world
22 pipetest: Test 3 -- PASS
23 pipetest: ALL TESTS PASSED
```

三个测试全部通过：

- Test 1: 父子进程双向 ping-pong 通信正常
- Test 2: 768 字节大块数据通过管道环形缓冲区（512 字节）正确回绕传输
- Test 3: 连续 5 条消息通过 \n 分隔符正确分帧

4.4 Shell PATH Fallback 测试

Listing 15: Shell 子目录执行命令测试

```
1 $ mkdir test
2 $ cd test
3 $ ls                                # 之前: exec ls failed
4 .                                  1 1 1024    # 现在: 正常工作
5 ..                                 1 1 1024
6
7 $ cat README                       # 之前: exec cat failed
8 [README 内容]                     # 现在: 正常工作
9
10 $ echo hello > new.txt            # 之前: open failed
11 $ cat new.txt                     # 现在: 正常工作
12 hello
```

4.5 UDP 网络通信测试

4.5.1 测试环境

宿主机终端启动监听：

```
1 $ nc -u -l 2000
```

4.5.2 Test 1: xv6 发送 → 宿主机接收

Listing 16: udptest 发送测试 (xv6 端)

```
1 $ udptest
2 udptest: socket fd=3
3 udptest: sending to 10.0.2.2:2000 ...
4 net: ARP who-has 10.0.2.2
5 net: ARP request sent for 10.0.2.2, ret=42
6 net: ARP recv from IP 10.0.2.2 MAC 52:55:a:0:2:2
7 net: ARP resolved after 0 tries
8 udptest: sent 15 bytes
9 udptest: waiting for reply ...
```

宿主机 nc -u -l 2000 收到 Hello from xv6!。

4.5.3 Test 2: 宿主机回复 → xv6 接收

Listing 17: udptest 接收回复测试

```
1 udptest: reply from 10.0.2.2:2000 = "Hello from host!"
```

4.6 调度算法测试（涂凡琛）

4.6.1 FCFS 测试

fcfstest 创建 4 个带间隔启动的 CPU 密集子进程，退出顺序与 PID/创建顺序一致，验证了 FCFS 先来先服务行为。

4.6.2 MLFQ 测试

mlfqtest 混合 SHORT / LONG / MIXED 三类作业，呈现 $SHORT < LONG < MIXED$ 的完成时间趋势，验证了 MLFQ 短作业优先特性。

4.6.3 信号量测试

表 7: 信号量测试结果

测试程序	状态	验证内容
semtest1	✓ PASSED	基本 P/V 操作、进程阻塞与唤醒
semtest2	✓ PASSED	4 进程 × 100 次操作 = 400 次互斥协调
semtest3	✓ PASSED	2 生产者 + 2 消费者，3 信号量协同

4.7 内存分配器测试（韩硕）

kmalloctest（启动时自动执行）覆盖：

- 边界值检查（0、2049）
- 基本分配（1、32、33、2000）
- 读写测试
- 复用测试
- 跨页扩展（连续 140 个 32B 对象）
- 大对象分配/释放（2049、3000、边界最大值）
- 空 slab 页归还（4 个 slab 页正确归还）

另外，启动阶段还会输出运行时物理内存探测结果，例如：

```
1 memory: [0x80000000, 0x88000000) size=0x8000000 (dtb)
2 memory: [0x80000000, 0x90000000) size=0x10000000 (dtb)
```

这说明系统已经能够随 QEMU -m 参数变化，自动识别并管理实际提供的 RAM 规模。综合 kmalloctest、页表路径和缺页分配逻辑验证，内存管理主路径运行稳定，无 panic。

4.8 文件管理测试（韩硕）

文件管理相关能力通过系统调用路径和专门测试程序共同验证：

- open/close/read/write：验证普通文件创建、打开、关闭和顺序读写
- mkdir/unlink/chdir：验证目录创建、空目录删除以及相对路径/绝对路径切换

- `dup/fork/exit`: 验证文件描述符共享、继承与自动回收
- `lseektest`: 验证 `SEEK_SET/SEEK_CUR/SEEK_END` 三种定位模式

其中 `lseektest` 已在 xv6 内实际运行通过, 说明文件偏移管理、读写路径和错误边界处理已经具备可用性。

五、 创新点

本项目的创新点主要体现在以下几个方面：

5.1 UDP 网络协议栈（侯奕明）

在 xv6 这样精简的教学内核中实现了完整的 UDP 网络协议栈是本项目最突出的创新之一。具体创新包括：

- **四层协议完整实现**：从硬件驱动(virtio-net MMIO)到协议栈(Ethernet/ARP/IP/UDP)再到 socket 系统调用接口，实现了完整的数据收发路径；
- **E1000→virtio-net 技术路线的发现与验证**：经过深入调试，发现了 QEMU 8.2.2 RISC-V 上 E1000 PCIe DMA 的 bug，并选择 virtio-net MMIO 作为替代方案，这一发现对后续在 QEMU RISC-V 上做网络开发具有参考价值；
- **ARP 自动解析机制**：实现了广播 ARP 请求 → 轮询等待 → 自动记录 → 超时重试的完整地址解析流程；
- **Socket 与文件系统的统一**：通过扩展 FD_SOCK 类型，将 socket 无缝集成到 xv6 的文件描述符体系中。

5.2 控制台输出原子化的两层协同方案（侯奕明）

针对多进程并发输出交错这一经典问题，提出了用户态动态缓冲 + 内核态 sleep lock 的两层协同方案：

- 用户态动态缓冲确保一次 printf = 一次 write，且支持自动扩容；
- 内核态 sleep lock 确保一次 write 不被其他进程插入；
- 两层保护协同使每次 printf 成为原子操作，且 sleep lock 选择正确（不违反 xv6 的 spinlock 不可 sleep 规则）。

5.3 QEMU Test Device 关机方案（侯奕明）

在发现 SBI ecall 在 -bios none 场景下不可用后，创新性地使用 QEMU virt 机器内置的 SiFive test device (0x100000) 实现关机。这一方案的优点是：

- 不依赖 SBI 固件；
- 仅需 3 个文件的修改 (memlayout.h 定义地址 + vm.c 添加映射 + sysproc.c 写寄存器)；
- 工作可靠，在任何情况下都能正常退出 QEMU。

5.4 两级内存分配器（韩硕）

在 xv6 原有只支持页级分配的基础上，实现了 size-class slab + 大对象回退 + 空页自动归还的完整内核堆分配器，并将 `struct pipe` 和 `struct file` 从整页浪费优化为精确分配。

5.5 多调度算法框架（涂凡琛）

通过编译期宏选择实现 RR/FCFS/MLFQ 三种调度算法的切换框架，以及将 `wakeup()` 从 $O(N)$ 优化为 $O(k)$ 的哈希等待队列设计。

六、 总结与展望

6.1 项目总结

本项目以 xv6-riscv 为基础平台，成功完成了《操作系统课程设计》A 题“OS 内核实现”的全部要求，并在多个方面进行了创新扩展。

侯奕明（接口、设备与软件交互） 完成了：

- 系统调用全链路分析与文档（ecall → usertrap → syscall → sret）
- 中断处理两条路径分析（用户态 vs 内核态被中断）
- 新增 7 个系统调用：ps（#22）、halt（#23）、lseek（#24）、socket（#23）/ bind（#24）/ sendto（#25）/ recvfrom（#26）
- 控制台输出原子化（内核 sleep lock + 用户态动态缓冲）
- Shell PATH fallback 机制
- halt shell 内建命令
- 管道通信测试程序 pipetest（3 类场景全覆盖）
- lseek 测试程序 lseektest（4 种模式全覆盖）
- **UDP 网络协议栈完整实现**：virtio-net MMIO 驱动 + Ethernet/ARP/IP/UDP 协议栈 + socket 接口
- udptest 网络测试程序（发送/接收/双向通信）

涂凡琛（进程管理与处理机调度） 完成了：

- FCFS + MLFQ 两种新调度算法实现
- 信号量机制（P/V 操作 + 3 个系统调用 + 3 组测试）
- wakeup 哈希等待队列优化（ $O(N) \rightarrow O(k)$ ）
- 多组调度测试程序（fcfstest/mlfqtest/throughput/csw）

韩硕（系统启动、内存管理与文件管理） 完成了：

- 系统启动链路实现与分析（_entry → start() → main()）
- 启动栈指针初始化、M 态到 S 态切换与 trap vector 初始化
- 启动时物理内存探测（DTB /memory 解析）与运行时 RAM 上界管理

- 基于空闲链表的物理页框分配器分析与 4KB 页管理
- 内核堆分配器 kmalloc/kmfree (slab + 大对象 + 空页归还)
- struct pipe 和 struct file 从整页分配优化为精确分配
- Sv39 虚拟内存管理与 lazy sbrk 缺页分配分析
- 文件描述符表管理、路径解析、文件/目录操作与 lseek 支持

团队整体完成了 xv6 环境搭建、编译运行验证、模块联调、文档撰写和汇报准备。

6.2 未来展望

尽管本项目已经完成了大量工作，仍有许多可以继续深入的方向：

6.2.1 网络协议栈增强（侯奕明）

- 实现 TCP 协议支持（需要更复杂的重传、拥塞控制、窗口管理机制）
- 实现 DHCP 客户端，支持动态 IP 获取
- 添加 UDP 校验和完整实现
- 实现非阻塞 socket 和 select/poll 多路复用
- ARP 表超时机制和定期清理
- 使用 QEMU tap 网络替代 SLIRP，支持外部网络访问

6.2.2 系统调用扩展

- 新增更多系统调用（如 sys_meminfo 内存状态观测）
- 实现 mmap/munmap 内存映射接口
- 增加更全面的系统调用测试集

6.2.3 调度与同步增强

- 实现更精细的 MLFQ 可观测性（队列迁移日志）
- 增加更多同步原语（读写锁、条件变量）
- 实现共享内存机制

6.2.4 文件系统增强

- 支持多级目录和符号链接
- 实现文件权限管理
- 增加文件系统一致性检查工具

6.3 个人收获（侯奕明）

通过本次课程设计，我在以下方面获得了显著的提升：

1. **对操作系统内核的深入理解**：从用户态系统调用到内核态 trap 处理再到设备中断的完整链路，建立起对操作系统”桥梁”角色的完整认知；
2. **RISC-V 架构的实践掌握**：ecall/sret 指令、CSR 寄存器（stvec/sepc/scause/stval/sstatus）、Sv39 页表、PLIC 中断控制器等；
3. **网络协议栈的从零实现经验**：从 QEMU 设备树解析、virtio MMIO 寄存器操作，到 Ethernet/ARP/IP/UDP 逐层构建，再到 socket 系统调用集成，经历了完整的网络栈开发流程；
4. **底层调试能力**：PCIe DMA bug 的定位、内核 panic 的逐层排查、设备地址的 DTB 解析、管道字节流特性导致的隐蔽 bug 等，大幅提升了系统级调试能力；
5. **工程能力与文档规范**：学会了使用 LaTeX 撰写规范的技术文档，掌握了结构化分析和系统化记录工程进展的方法。

总的来说，本次课程设计不仅让我们完成了一个可运行、可展示的操作系统内核扩展项目，更重要的是建立了对操作系统内核的深度理解、培养了系统级开发与调试的工程能力。